Nitin Avula

+1 (352) 219-4654 | nitina2505@gmail.com | linkedin.com/in/nitin-a-312980333 | github.com/Nitin172503 | nitin172503.github.io/nitin-portfolio

EDUCATION

University of Florida **Aug 2024 – May 2026**
M.S. Computer & Information Science & Engineering GPA: 3.7/4.0 *Gainesville, FL*

VIT-AP University Aug 2020 – May 2024
B.Tech, Computer Science & Business Systems

Software engineer (MS, University of Florida, May 2026) building distributed systems, AI-powered backends, and production APIs. I measure first, change one thing, measure again. Comfortable at the seam between backend systems and ML. Available immediately, open to relocation.

TECHNICAL SKILLS

Languages: Go, Python, Java, TypeScript/JavaScript, C++, SQL
Backend & APIs: Spring Boot, FastAPI, Flask, Node.js/Express, microservices, event-driven architecture, Kafka, REST APIs
AI & ML: LLM APIs (Anthropic, OpenAI, Gemini/Ollama), RAG pipelines, context-window management, structured outputs, Random Forest, Gradient Boosting, Pandas/NumPy
Data: PostgreSQL, MySQL, MongoDB, SQL query optimization, index tuning, execution-plan analysis
Cloud & DevOps: AWS (EC2, CloudWatch), Docker, Kubernetes, Prometheus, Grafana, Jenkins, GitHub Actions, CI/CD

EXPERIENCE

Iconize Technologies **Oct 2023 – Jun 2024**
Software Engineer Intern *Gainesville, FL*

- Built backend microservices for a genomic analytics platform with modular RESTful APIs, improving researcher query turnaround by 30%.
- Optimized PostgreSQL via index tuning, execution-plan analysis, and connection pooling, cutting average API response latency by 40%.
- Architected resilient workflow orchestration with validation, retry handling, and fault isolation, lowering pipeline failure rates by 25%.
- Automated containerized deployments with Jenkins, rolling updates, and health monitoring, cutting deploy time from 20 to 6 minutes.
- Performed root-cause analysis and production debugging; authored API and architecture documentation that accelerated team onboarding.

SmartInternz **May 2023 – Jul 2023**
Data Science Intern

- Delivered a production traffic-prediction system with ensemble learning (Random Forest + Gradient Boosting) reaching 92% accuracy.
- Deployed a low-latency inference service handling concurrent requests with memory-aware batching, sustaining P95 under 200ms.
- Built scalable data-preprocessing pipelines in SQL and Python to clean, transform, and validate high-volume datasets.
- Established real-time observability with AWS CloudWatch metrics, logs, and alarms, lowering anomaly detection time by 35%.

PROJECTS

Distributed Task Queue — *Go, Redis, Prometheus, Grafana, Docker, Lua*

- Priority-based distributed task queue with a pluggable broker (in-memory or Redis — same application code), concurrent worker pool, and REST API. Crash-safe by design: Redis dequeue is a single Lua script that atomically moves a task from pending into a processing ZSet — a task is never lost between "popped" and "marked processing." A background visibility-timeout reaper reclaims tasks whose workers died or hung.
- Full observability stack — Prometheus metrics, alert rules, and Grafana dashboard auto-provisioned via docker-compose. Operational hardening includes bearer-token auth, backpressure (queue-depth cap → 429), and a dead-letter path with an explicit retry endpoint.

AI Software Engineering Copilot — *Go, React, TypeScript, LLM APIs, Docker, Ollama/Gemini*

- Upload a GitHub repo or ZIP and get an LLM-backed engineering review: architecture overview, generated docs, bug findings, refactor suggestions, and unit tests. Bounded context-window management for large repos; zip-slip-hardened file handling; structured JSON output defensively parsed with graceful fallback.
- LLM behind a one-method interface with two live implementations (Ollama offline, Gemini public demo) selected at runtime. Upload caps, clone timeouts, concurrency limiter, and TTL janitor for temp directories.

Microservice Transaction System — *Java, Spring Boot, Kafka, PostgreSQL*

- Async transaction processor on Spring Boot and Kafka sustaining 50 events/sec under load; idempotent consumers, transactional writes, and persistent storage. Integration tests covering failure recovery, replay, and out-of-order event handling.